Sheep Compendium

Test-driven development activity · FastAPI and pytest

GitHub: https://github.com/reay0110/sheep_compendium

This project is a small farmer's inventory of sheep. Each sheep has an id, name, breed, and sex. The tests were written with pytest and FastAPI's TestClient, and the routes were filled in so those tests would pass. The flock is stored in a Python dictionary, not a SQL database, which is what the lab uses so the work stays on the tests and the endpoints.

1. Adding a sheep — completed test

The new sheep is Marigold, the Suffolk ewe from the handout. The lab note says to send the body with `json=sheep_data_name`, so the dictionary uses that name. The test posts to `/sheep/`, expects status 201, checks that the JSON body matches what was sent, and then reads the same id back to prove the record was stored.

tests/test_main.py

```python
# Define a test function for adding a new sheep
def test_add_sheep():
    # Prepare the new sheep data in a dictionary format.
    # Marigold is the Suffolk ewe from the lab handout.
    sheep_data_name = {
        "id": 7,
        "name": "Marigold",
        "breed": "Suffolk",
        "sex": "ewe",
    }

    # Send a POST request to the endpoint "/sheep/" with the new sheep data.
    # Arguments should be your endpoint and new sheep data.
    response = client.post("/sheep/", json=sheep_data_name)

    # Assert that the response status code is 201 (Created)
    assert response.status_code == 201

    # Assert that the response JSON matches the new sheep data
    assert response.json() == sheep_data_name

    # Verify that the sheep was actually added to the database by retrieving the new sheep by ID.
    # Include an assert statement to see if the new sheep data can be retrieved.
    get_response = client.get(f"/sheep/{sheep_data_name['id']}")
    assert get_response.status_code == 200
    assert get_response.json() == sheep_data_name
```

Figure 1. `tests/test_main.py` — `test_add_sheep`

2. The test fails before the route exists

Following the lab, this test was run before `POST /sheep/` was added. Reading Spice still passed, because that route was already in place. Adding Marigold failed: the server answered 404, and the test expected 201.

```
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\reger\Projects\sheep_compendium
plugins: anyio-4.15.1
collected 2 items

tests\test_main.py .F                                                    [100%]

================================== FAILURES ===================================
_____________________________ test_add_sheep ______________________________
tests\test_main.py:51: in test_add_sheep
    assert response.status_code == 201
E   assert 404 == 201
E    +  where 404 = <Response [404 Not Found]>.status_code
=========================== short test summary info ===========================
FAILED tests/test_main.py::test_add_sheep - assert 404 == 201
========================= 1 failed, 1 passed in 0.90s =========================
```

Figure 2. pytest before the add endpoint: 1 failed, 1 passed. `assert 404 == 201`

3. The add endpoint

`models/models.py` defines a Pydantic `Sheep` model with id, name, breed, and sex. `models/db.py` keeps those records in `FakeDB.data`. `add_sheep` refuses a duplicate id by raising `ValueError`. The route in `main.py` does the same check and returns HTTP 400 instead, then stores the sheep and responds with 201. That is the status the test asserts.

main.py

```python
from fastapi import FastAPI, HTTPException, status

from models.db import db
from models.models import Sheep

app = FastAPI()


@app.get("/sheep/", response_model=list[Sheep])
def read_all_sheep():
    return db.get_all_sheep()


@app.get("/sheep/{id}", response_model=Sheep)
def read_sheep(id: int):
    sheep = db.get_sheep(id)
    if sheep is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Sheep not found")
    return sheep


@app.post("/sheep/", response_model=Sheep, status_code=status.HTTP_201_CREATED)
def add_sheep(sheep: Sheep):
    # Check if the sheep ID already exists to avoid duplicates
    if sheep.id in db.data:
        raise HTTPException(status_code=400, detail="Sheep with this ID already exists")

    # Add the new sheep to the database
    db.data[sheep.id] = sheep
    return sheep  # Return the newly added sheep data


@app.put("/sheep/{id}", response_model=Sheep)
def update_sheep(id: int, sheep: Sheep):
    try:
        return db.update_sheep(id, sheep)
    except ValueError:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Sheep not found")


@app.delete("/sheep/{id}", response_model=Sheep)
def delete_sheep(id: int):
    try:
        return db.delete_sheep(id)
    except ValueError:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Sheep not found")
```

Figure 3. `main.py` after the required route and the extra-credit routes

With the route in place, the same two tests pass.

```
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\reger\Projects\sheep_compendium
plugins: anyio-4.15.1
collected 2 items

tests\test_main.py ..                                                    [100%]

============================== 2 passed in 0.80s ==============================
```

Figure 4. pytest after POST `/sheep/` was added: 2 passed

4. Swagger UI

The app was started with `uvicorn main:app`. Port 8000 was already taken by another local project, so this server used `http://127.0.0.1:8001/docs`. The screenshots below are from that page's Try it out panel.

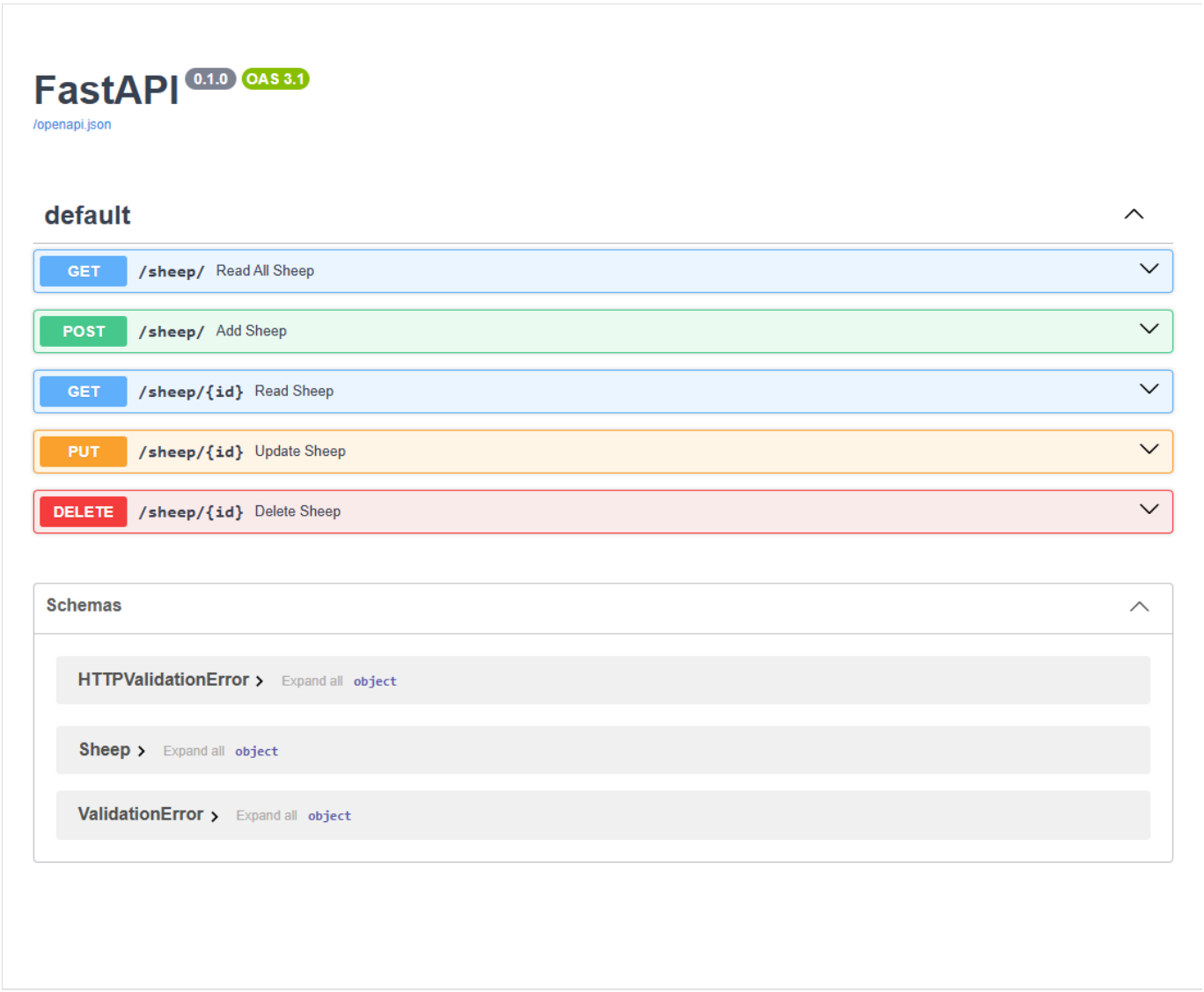


Figure 5. The five routes: list, add, read one, update, and delete

GET /sheep/{id} Read Sheep

Parameters Cancel

Name	Description
id * required integer (path)	1

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/sheep/1' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8001/sheep/1
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "name": "Spice",
  "breed": "Gotland",
  "sex": "ewe"
}
```

Download

Response headers

```
content-length: 53
content-type: application/json
date: Sat,31 Oct 2026 00:52:12 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type: application/json

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "breed": "string",
  "sex": "string"
}
```

Code	Description	Links
422	Validation Error	*No links*

Media type: application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
```

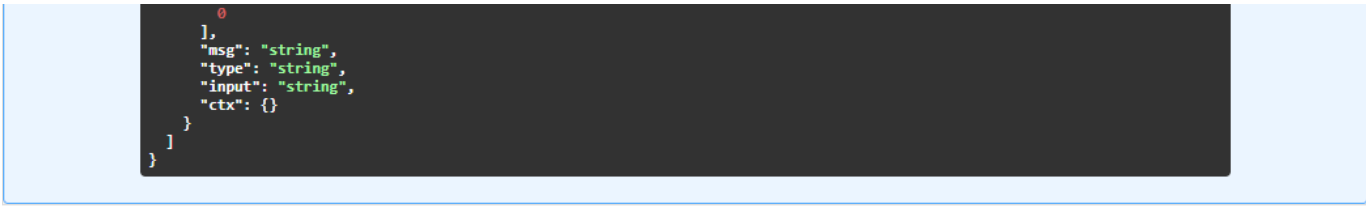


Figure 6. GET /sheep/1 returns Spice, the Gotland ewe, with status 200

POST /sheep/ Add Sheep

Parameters | Cancel | Reset

No parameters

Request body required | application/json

Edit Value | Schema

```
{
  "id": 7,
  "name": "Marigold",
  "breed": "Suffolk",
  "sex": "ewe"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/sheep/' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 7,
  "name": "Marigold",
  "breed": "Suffolk",
  "sex": "ewe"
}'
```

Request URL

```
http://127.0.0.1:8001/sheep/
```

Server response

Code	Details
201	Response body

```
{
  "id": 7,
  "name": "Marigold",
  "breed": "Suffolk",
  "sex": "ewe"
}
```

Download

Response headers

```
content-length: 56
content-type: application/json
date: Sat,31 Oct 2026 00:52:15 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	*No links*

Media type



Figure 7. POST /sheep/ creates Marigold and returns 201

5. Extra credit

`update_sheep` takes the id and the new Sheep record separately. `add_sheep` only takes a Sheep and uses `sheep.id` as the key. A missing id becomes a 404. Delete removes the record and returns it. List returns every value in the dictionary.

Read all sheep

The test asks for `GET /sheep/`, checks that the body is a list, and checks that the original six ids are present, including Spice's record. It does not require an exact length, so it still passes after another test has added a sheep.

tests/test_main.py

```python
def test_read_all_sheep():
    response = client.get("/sheep/")
    assert response.status_code == 200

    sheep = response.json()
    assert isinstance(sheep, list)
    by_id = {item["id"]: item for item in sheep}
    assert {1, 2, 3, 4, 5, 6}.issubset(by_id)

    assert by_id[1] == {
        "id": 1,
        "name": "Spice",
        "breed": "Gotland",
        "sex": "ewe",
    }
```

Figure 8. `test_read_all_sheep`

Code | **Details**

200

Response body


```
[
  {
    "id": 1,
    "name": "Spice",
    "breed": "Gotland",
    "sex": "ewe"
  },
  {
    "id": 2,
    "name": "Blondie",
    "breed": "Polypay",
    "sex": "ram"
  },
  {
    "id": 3,
    "name": "Deedee",
    "breed": "Jacobs Four Horns",
    "sex": "ram"
  },
  {
    "id": 4,
    "name": "Rommy",
    "breed": "Romney",
    "sex": "ram"
  },
  {
    "id": 5,
    "name": "Vala",
    "breed": "Valais Blacknose",
    "sex": "ewe"
  },
  {
    "id": 6,
    "name": "Esther",
    "breed": "Border Leicester",
    "sex": "ewe"
  },
  {
    "id": 7,
    "name": "Marigold",
    "breed": "Suffolk",
    "sex": "ewe"
  }
]
```


Download

Response headers

```
content-length: 412
content-type: application/json
date: Sat,31 Oct 2026 00:54:46 GMT
server: uvicorn
```

Figure 9. GET /sheep/ after Marigold was added. Rommy's sex is ram here because the update in Figure 11 had already been sent.

Update a sheep

Blondie's seeded record is a ram. The test changes that sex to ewe and reads the id again to confirm the write. In Swagger, the same route was used on Rommy, changing the seeded ewe record to a ram.

tests/test_main.py

```python
def test_update_sheep():
    # Blondie is stored as a ram. This request changes the sex and checks the write stuck.
    updated = {
        "id": 2,
        "name": "Blondie",
        "breed": "Polypay",
        "sex": "ewe",
    }
    response = client.put("/sheep/2", json=updated)
    assert response.status_code == 200
    assert response.json() == updated

    get_response = client.get("/sheep/2")
    assert get_response.status_code == 200
    assert get_response.json() == updated
```

Figure 10. `test_update_sheep`

PUT `/sheep/{id}` Update Sheep

Parameters | Cancel | Reset

Name	Description
id * required integer *(path)*	4

Request body required — application/json

Edit Value | Schema

```
{
  "id": 4,
  "name": "Rommy",
  "breed": "Romney",
  "sex": "ram"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8001/sheep/4' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 4,
  "name": "Rommy",
  "breed": "Romney",
  "sex": "ram"
}'
```

Request URL

```
http://127.0.0.1:8001/sheep/4
```

Server response

Code	Details
200	Response body

```
{
  "id": 4,
  "name": "Rommy",
  "breed": "Romney",
  "sex": "ram"
}
```

Download

Response headers

```
content-length: 52
content-type: application/json
date: Sat,31 Oct 2026 00:52:19 GMT
server: uvicorn
```

Responses

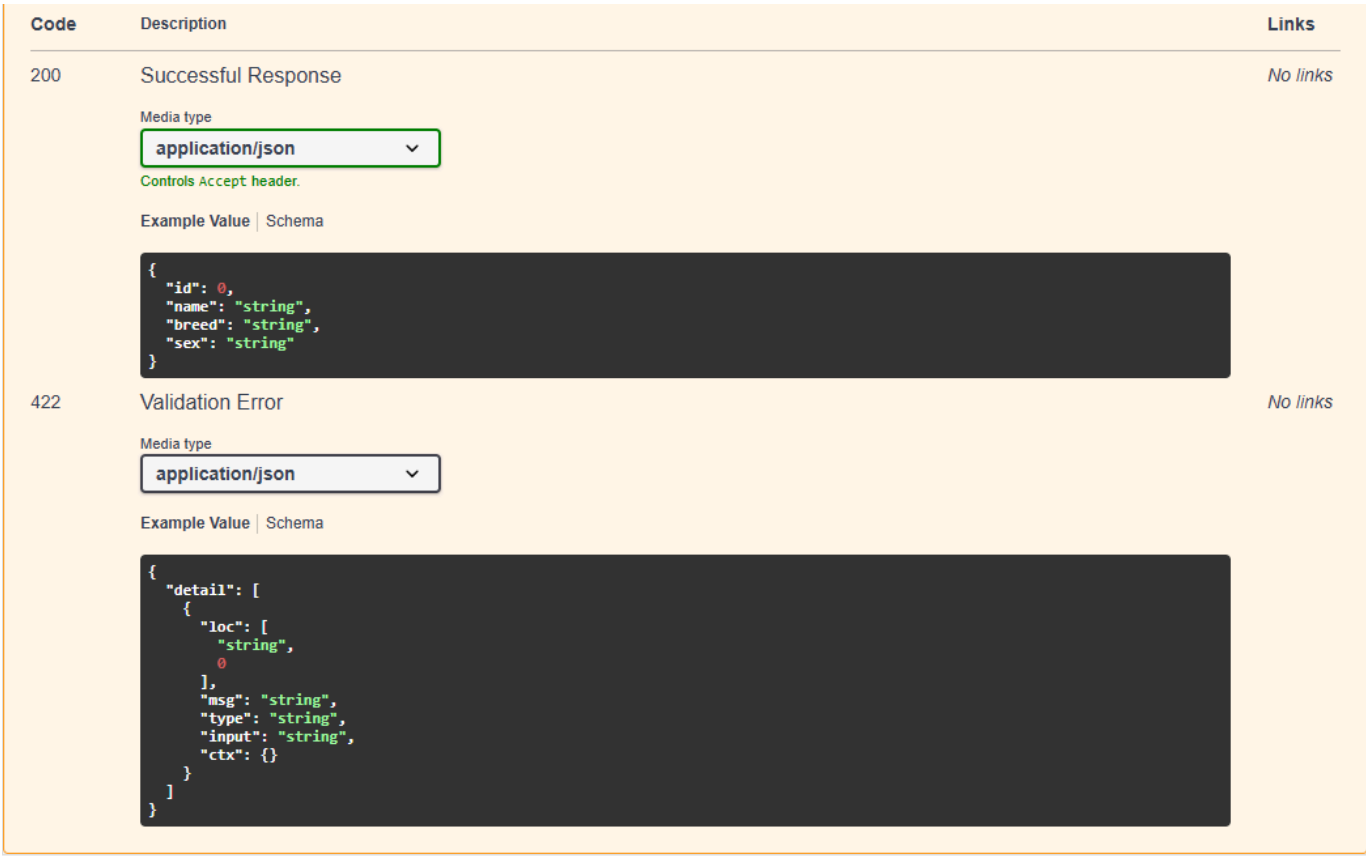


Figure 11. PUT /sheep/4 updates Rommy and returns 200

Delete a sheep

Button is the Babydoll from the handout. The test creates him, deletes him, and checks that a later GET returns 404. The read route returns 404 when `get_sheep` finds nothing, which is what makes that last assert possible. Swagger shows the same delete for Marigold.

tests/test_main.py

```python
def test_delete_sheep():
    # Button is the Babydoll from the lab handout. Create it, then remove it.
    sheep_data_name = {
        "id": 8,
        "name": "Button",
        "breed": "Babydoll",
        "sex": "ram",
    }
    created = client.post("/sheep/", json=sheep_data_name)
    assert created.status_code == 201

    response = client.delete("/sheep/8")
    assert response.status_code == 200
    assert response.json() == sheep_data_name

    missing = client.get("/sheep/8")
    assert missing.status_code == 404
```

Figure 12. `test_delete_sheep`

DELETE /sheep/{id} Delete Sheep

Parameters Cancel

Name	Description
id * required integer (path)	7

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8001/sheep/7' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8001/sheep/7
```

Server response

Code	Details
200	Response body

```
{
  "id": 7,
  "name": "Marigold",
  "breed": "Suffolk",
  "sex": "ewe"
}
```

Download

Response headers

```
content-length: 56
content-type: application/json
date: Sat,31 Oct 2026 00:52:22 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "breed": "string",
  "sex": "string"
}
```

Code	Description	Links
422	Validation Error	*No links*

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
```

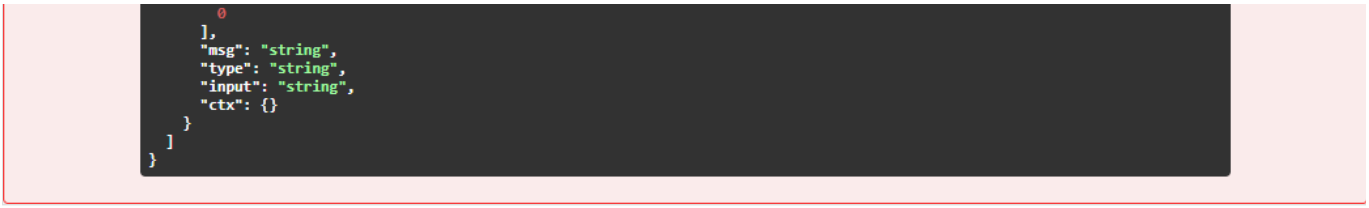


Figure 13. DELETE /sheep/7 removes Marigold and returns the deleted record

Full suite

The extra tests cover a duplicate add (400), a missing update (404), and a missing delete (404), along with the three new routes. All eight tests pass.

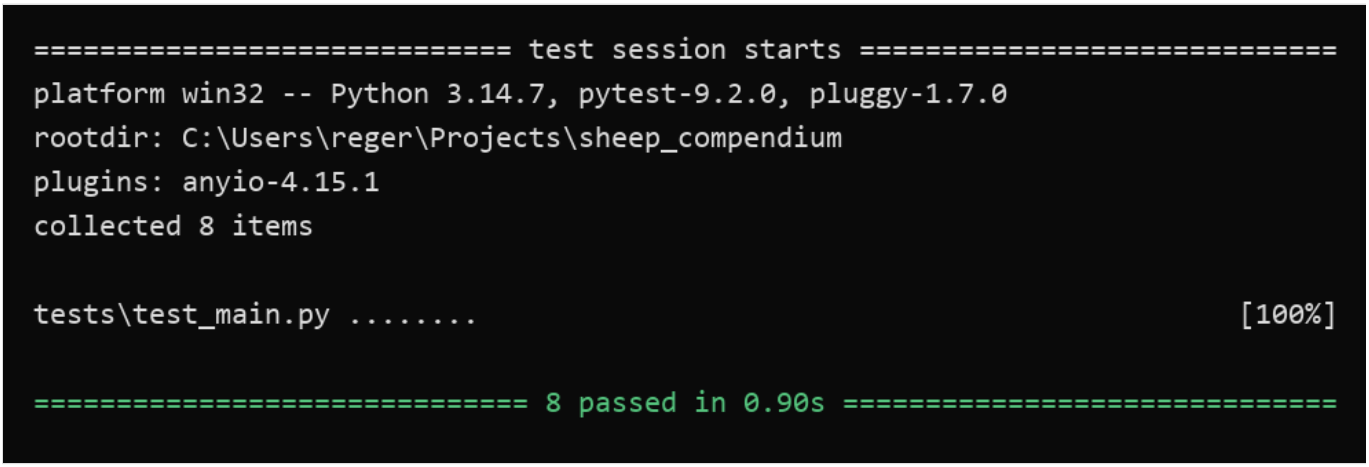


Figure 14. Full pytest run: 8 passed

How to run it

From the project folder:

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m pytest
.venv\Scripts\python -m uvicorn main:app --reload
```

Then open `http://127.0.0.1:8000/docs`.